D2 Detailed Design — Smart Closet Mirror

Assignment #4 (Design Diagrams), part D2 · Team bullriders · Arya Garg, Clare Hooper, Rehman Khalid, Jace Schubert · Week 7

1. Header, conventions, and D2 overview

Project title: Smart Closet Mirror

> **Goal statement.** Turn a Gen 1 Mirror into a closet mirror that measures the user with a Kinect, shows their own clothes and shop items on their body, and tells them what to wear today and whether a new piece is worth buying.

Overview for a reviewer with no context (two minutes)

What the system is. A Gen 1 Mirror (an Android 6 smart mirror) becomes the screen for a closet assistant. A Kinect 360 on top of the Mirror sees the user in colour and depth. A Ubuntu PC in the home (the "brain") does all the work in eight layers, L1–L8 (D0, Fig 1): capture, perception, body, wardrobe, try-on, decision, connectors to outside services, and presentation. The Kinect stays off until the person at the glass allows it for that session. The user adds clothes by photographing them with a phone. The Mirror shows them on the user's body in live video and tells them what to wear today and whether a shop item is worth buying.

What D1 covered. D1 detailed the two riskiest components. L4 Wardrobe has the main data model (D1 Fig 1) and the phone API. L5 Try-on has the fit check (algorithm A1) and the live garment drawing (algorithm A2).

What D2 adds.

- **Changes driven by the Week 4 user stories**, marked [US-xx] where they appear: current weather in outfit suggestions (US-01), camera capture only after consent (US-04), a profile PIN so each person sees only their own wardrobe (US-05), and configuration rollback for maintainers (US-03).
- Every remaining component (L1, L2, L3, L6, L7, L8) and the shared planes (data store, message bus) now has a data model, behaviour, and interface (part 2).
- The four glue components get the same treatment: authentication, notifications, configuration, and logging and health.
- Three state machines: the Mirror session (Fig D2-1), body measurement (Fig D2-2), and camera consent (Fig D2-5).
- Two flows that show their error paths: "buy or skip?" (Fig D2-3) and "recommend an outfit" (Fig D2-6).
- The data model for the new tables (Fig D2-4).
- Revisions to five D0 interfaces that had grown too loose (part 2, first table).
- The edge-case table, requirements traceability, design rationale, and peer-review notes (parts 3–6).

Known gaps and open questions

#	Gap or question	Current position

G1	No shop product API or image search provider has been chosen yet (I14b, I14c).	L7 uses one adapter per provider, so the choice does not affect other layers. Candidates must allow non-commercial student use without fees. *[team: pick by Week 8]*
G2	A profile PIN typed on the glass can be watched by someone standing nearby (US-05).	The keypad shuffles its digits on every unlock and does not echo them. The PIN protects wardrobe privacy, not payments. Unlocking from the paired phone is a possible later improvement.
G3	Accuracy of body circumferences from one front and one side depth view (A3) is unknown.	Target ±2 cm against a tape measure on all four team members before Week 9. The calibration factors live in the configuration file.
G4	Phone notifications are in-app only (no push), because push services would send data off the home network.	The phone page polls when open; the glass shows toasts. Accepted as a privacy trade-off.
G5	Which machine is the brain PC, and does it have an NVIDIA GPU (needed for A2 and NVENC)?	Fallback: libx264 software encoding at 20 fps. *[team: confirm]*
G6	Kinect 360 depth quality under the Mirror's own lighting and at 1–3 m in a typical room.	To be tested in the Week 8 hardware spike.
G7	The traceability table (part 4) lists the five user stories but the mapping is not yet filled in.	*[team: complete part 4]*
G8	The Week 4 user stories call the project "Smart Mirror Personal Stylist"; D0–D2 use "Smart Closet Mirror".	*[team: choose one title and use it in every document and diagram]*
G9	The user stories do not mention try-on, body measurement, buy-or-skip, or looks from photos.	The reverse check in part 4 must either tie L3, L5, A3, A5, and A7 to a requirement or record them as scope to cut. *[team decision]*

Conventions for every D2 figure

Each figure carries the project title, the goal statement, and its own legend. The legend on the figure is the authority; this table repeats it. D0 and D1 figures keep the conventions stated in those documents.

Figure	Boxes and shapes	Lines
Fig D2-1 Mirror session (state machine) Fig D2-2 Body measurement (state machine) Fig D2-5 Camera consent (state machine)	Rounded box = state. Dashed square box = error state. Thick box = final (success) state. Black dot = start state. Large dashed rounded outline = a group of states.	Arrow = transition, labelled "trigger" or "trigger / action"; text in [brackets] is a guard that must be true. Dashed arrow = error or timeout transition. An arrow leaving a group applies to every state inside it.
Fig D2-3 "Buy or skip?" with error paths (flow) Fig D2-6 "Recommend an outfit" with error paths (flow)	Rounded box = step, naming the component and interface. Diamond = check. Thick box = successful end. Dashed box = error outcome, naming the decided behaviour (retry, degrade, fall back, surface error) and the handler.	Solid arrow = happy path. Dashed arrow = error path. Labels on arrows are check outcomes (yes / no).
Fig D2-4 Data model (entity-relationship)	Box = entity (table); rows = key attributes with types. Dashed box = entity defined in D1 Fig 1. Keys: PK = primary key (underlined), FK = foreign key, UQ = unique, UQ1 = unique together, IX = indexed.	Line = relationship; the italic verb reads from the first-named entity. Cardinality at both ends in crow's-foot notation, repeated as text: 1, 0..1, 1..*, 0..*.

Interface IDs continue D0's numbering (I1–I17). D2 adds I18–I28 and splits I14 into I14a–d (see "Interface revisions"). Algorithm IDs continue D1's: A1–A2 in D1, A3–A7 here. Edge-case IDs are EC-<component>-<category>, for example EC-L7-X.

2. Finished components (the D2 diagrams)

D1 already finished L4 Wardrobe and L5 Try-on. This part finishes everything else. Each component lists its **data model**, its **behaviour**, and its **interface** with errors. Every error, on the bus or over HTTP, uses the D1 shape {"error": {"code", "message", "fields"}}.

Interface revisions (D0 → D2)

ID	D0 definition	Problem	D2 revision
I6 → I6 + I18	L8 → WebView: ui_state and control on one channel.	Mixed two concerns: screen content (idempotent, resendable) and device commands (not idempotent, such as waking the glass).	**I6 ui.state** (WebSocket, full state with seq) and **I18 mirror.control** (JS bridge, one command per message with an ack).
I7	FrameBundle with raw shared-memory handles.	Leaked L1's memory layout; a consumer could read a buffer L1 was overwriting.	frame_bundle/2: ring slot 0–3 plus a generation counter. Consumers check the generation before and after reading and discard the frame if it changed.
I9	Avatar {mesh_ref glTF, rig}.	After D1 moved rendering to a 2-D warp (A2), nothing reads a 3-D mesh; the mesh was an internal L3 detail.	**BodyModel** {profile_id, version, rest_joints, proportions}, the only fields A1 and A2 use.
I14 → I14a–d	One ContextQuery{kind, params: map}.	Too loose: an untyped map meant callers and L7 could disagree on parameters, and errors could not be specific. It also had no weather, which US-01 needs.	Four typed methods: **I14a ctx.calendar_day**, **I14b ctx.products**, **I14c ctx.images**, **I14d ctx.weather** [US-01].
I5	Mirror camera + mic → L8: presence at 2 Hz and voice commands.	Presence came from watching the room through the Mirror's camera before anyone agreed, which US-04 forbids.	I5 carries only touch events and wake-word or voice commands. The Mirror's camera is never used [US-04].
I15 → I15 + I19	L6 ↔ L2 carried image analysis *and* gesture events.	Two different timings (a 5 s request vs a stream of events) and directions on one ID.	**I15 vision.analyze** (request/reply) and **I19 intent.gesture** (L2 → L6 events).
I8	BodyMeasurements.	L3 needed the T-pose joint positions to build the BodyModel and had no way to get them.	Adds rest_joints (20 × xyz, metres). Schema body_measurements/2.
New	Glue and control interfaces that D0 did not draw.		**I20** notify.create · **I21** ops.health · **I22** pairing API · **I23** config.changed · **I24** measure.start / cancel (L6 → L2) · **I25** measure.progress (L2 → L8) · **I26** notifications API (phone) · **I27** capture.enable / disable (L8 → L1) [US-04] · **I28** auth.unlock (L8 → Auth) [US-05].
D1 entities	Additions to D1 Fig 1 (D1 itself is unchanged; this table records them).		**Garment** gains warmth: INT 1–3 and water_resistant: BOOL, set when a photo is uploaded and editable by PATCH; A4 needs them [US-01]. **GET /api/v1/garments**: the optional owner parameter now accepts only the caller's own id; any other returns 403 FORBIDDEN [US-05].

L1 Capture — owner: Rehman Khalid

Data model: no persistent data. It holds only a 4-slot ring buffer in shared memory and its device settings from configuration.

Behaviour (steps; the device states are Disabled → Opening → Streaming → Stalled, with Disabled the default):

1. **Consent gate [US-04]:** start in Disabled with the Kinect closed and its LED off. Open the device only when L8 sends capture.enable (I27) for a session where the user said yes (Fig D2-5). On capture.disable, or when the session ends, close the device, turn the LED off, and confirm. Frames stay in shared memory; L1 has no code path that writes them to disk.
2. Open the Kinect with libfreenect (device index from config). Set RGB to 640 × 480 and depth to "registered" mode, so depth pixels line up with colour pixels.
3. The driver calls back once per RGB frame and once per depth frame. Pair the two when their timestamps are within 20 ms, and write the pair into the next ring slot. Increment that slot's generation.
4. Publish a FrameBundle (I7) with the slot number and generation. Every 5 s, publish health (I21) with the measured frame rate.
5. If no pair arrives for 2 s, go to Stalled: publish health DOWN, close the device, and try to reopen it every 5 s. When frames flow again, publish health OK.

Interface	Input	Output	Errors
I7 frames.raw (publishes)	—	frame_bundle/2 {seq: uint64, ts: int64 ns, slot: int 0–3, generation: uint64, width: 640, height: 480}	None sent; a consumer that sees the generation change discards the frame.
capture.status (bus request)	{}	{state: enum{disabled, opening, streaming, stalled}, device_open: bool, fps: float, last_frame_ts: int64 ns}	NO_SERVICE (bus) if L1 is not running; L8 treats this as "Camera status unknown".
I27 capture.enable / capture.disable (bus request, from L8)	{session_id: UUID, required; consent_id: UUID, required for enable}	{state: "streaming" | "disabled", device_open: bool}, sent only after L1 has checked the device	NO_CONSENT — consent_id unknown or already revoked · DEVICE_NOT_FOUND · TIMEOUT (3 s at the caller → Unknown, Fig D2-5)
I21 ops.health (publishes)	—	See Logging and health.	DEVICE_NOT_FOUND and USB_TRANSFER_ERROR are reported as detail, with state DOWN.

L2 Perception — owner: Arya Garg

Data model: no persistent data. Model weights are files listed in configuration. Measurements go to L3 (I8), which stores them.

Behaviour. L2 only receives frames while capture is on (Fig D2-5), so pose, gestures, and measurement are unavailable when the user has declined [US-04]. L2 runs four jobs in one process:

- **Pose (every frame, ≤ 60 ms, D0 budget):** run MediaPipe Pose on the RGB frame to get 33 landmarks, and map them to our 20 joints. Each joint's z is the median depth in a 5 × 5 window around it, ignoring zero pixels. Mask = MediaPipe segmentation AND depth within ±0.6 m of the torso. Publish I10, newest frame only.
- **Gestures:** rules over the smoothed joints.
  - A swipe is the right wrist moving more than 0.35 m sideways within 0.5 s at chest height: *next* or *back*.
  - A right hand held above the head for 1 s is *select*.
  - Only gestures with confidence ≥ 0.8 are published, at most one per second (I19).
- **Measurement:** algorithm A3, below and in Fig D2-2.
- **Item detection (I15):** run an open-vocabulary detector (OWLv2) with one text prompt per garment category. Keep boxes with confidence ≥ 0.3, and merge overlaps (IoU > 0.5). The dominant colour is k-means (k = 3) on the box pixels, named by the nearest of 16 palette colours in CIELAB.

Algorithm A3: body measurement from T-pose (state machine in Fig D2-2).

- **What it is:** it turns about 30 front and 30 side depth frames into body measurements in centimetres.
- **In:** FrameBundles plus LivePose for one user, with user_id and session_id.
- **Out:** BodyMeasurements (I8).
- **Steps:**
  1. Fit the floor plane once with RANSAC (200 iterations) on depth points below the hips.
  2. Height = distance from the floor to the highest mask point.
  3. At the chest, waist, and hip levels (set from the shoulder and hip joints), take the front width *w* = mask row width × depth / fx, and the side depth *d* the same way from the side frames.
  4. Circumference = the perimeter of an ellipse with axes *w* and *d* (Ramanujan's formula), times a calibration factor from configuration (gap G3).
  5. Shoulder width comes from the front mask at shoulder height. Inseam = hip-joint height above the floor minus a calibrated offset.
  6. Each value is the median across frames.
  7. Confidence = mean joint confidence × (1 − mean relative spread across frames), limited to 0–1.
- **Complexity:** O(P) per frame for P = 307,200 pixels, over about 60 frames, which takes under 1 s in total.

Interface	Input	Output	Errors
I24 measure.start / measure.cancel (request)	{schema: "measure_start/1", user_id: UUID, required; session_id: UUID, required}	{accepted: true, state: "wait_front"}	BUSY — a session is already running · NO_PERSON — nobody in view at start

I25 measure.progress (publishes)	—	{session_id, state: enum (Fig D2-2), hint: string ≤ 60, progress: float 0–1, reason: string?}	On failure, state = failed and reason ∈ {TIMEOUT, PERSON_LOST, MULTIPLE_PEOPLE, LOW_CONFIDENCE, OUT_OF_RANGE}.
I8 body.measured (publishes)	—	body_measurements/2: D0 fields plus rest_joints: 20 × {x, y, z: float m}	Only published when confidence ≥ 0.7.
I10 body.pose, **I19** intent.gesture (publishes)	—	I10 as in D0. I19 {user_id, intent: enum{next, back, select}, conf: float, ts}	None; when tracking is lost, I10 simply has no person.
I15 vision.analyze (request)	{request_id: UUID; image_refs: URI[], 1–6 items}	DetectedItems as in D0, plus color_name: string	TOO_MANY_IMAGES (> 6) · IMAGE_UNREADABLE (per image, in fields) · MODEL_UNAVAILABLE — weights missing or GPU out of memory · timeout 5 s at the caller

L3 Body — owner: Arya Garg

Data model: BodyProfile (D1 Fig 1) and BodyModel (Fig D2-4). Each profile has 0..* model versions, one per algorithm version.

Behaviour (steps on each I8 message):

1. Check the message again (defence in depth): confidence ≥ 0.7 and every value within human range. If either fails, log and drop it.
2. If the user's current profile differs by less than 1.0 cm in every measurement, keep it. Only update captured_at, so the fit cache stays valid.
3. Otherwise, in one transaction: insert a new BodyProfile, clear is_current on the old one, and set it on the new one.
4. Build the BodyModel: rest joints from I8, and proportions (torso, arm, and leg length; shoulder width in cm) from the joints.
5. Publish I9. L5's fit cache misses on its own, because the profile_id is new.

Interface	Input	Output	Errors
I9 body.model (publishes)	—	body_model/1 {user_id, profile_id, version: int, rest_joints: 20 × xyz m, proportions: {torso_cm, arm_cm, leg_cm, shoulder_cm}}	—
body.current (request)	{user_id: UUID, required}	The latest body_model/1 for the current profile	NO_CURRENT_PROFILE — the user has never been measured · UNKNOWN_USER

L6 Decision — owner: Clare Hooper

Data model: Outfit, OutfitItem, and WearEvent (D1 Fig 1), plus Decision, StylePreference, and SpendEntry (Fig D2-4). StylePreference holds the home location, rounded to 0.1°, for weather. Every query is scoped to the unlocked user [US-05].

- Decision keeps garment_ids as a JSON snapshot, so a past decision still reads correctly after a garment is archived.
- inputs_hash lets L6 reuse a decision when nothing has changed.
- Budget left this month = User.monthly_budget − the sum of this month's SpendEntry rows.

Behaviour. The five algorithm parts for A1/A2 are in D1. A4, A5, and A7 are given here as inputs, outputs, steps, and cost; their edge cases go in part 3.

- **A4 Outfit recommendation [US-01, UC-01]** (flow and error paths in Fig D2-6).
  - **In:**
    - occasion: from the request, else the first calendar event (I14a), else casual
    - closet index (I13) for the unlocked user only
    - weather (I14d)
    - StylePreference, WearEvents for the last 14 days, FitResults for the current profile
  - **Out:** a DecisionView of kind outfit_of_day with a top, a bottom (or a dress), shoes, and optional outerwear, plus a reason naming the occasion and the weather. Or "No complete outfit available" with the missing categories.
  - **Steps:**
    1. **Completeness first:** the available items must include a top and a bottom (or a dress), and shoes. If not, return "No complete outfit available" with the missing categories (for example, "bottom"). Never suggest an item that is not in the inventory (C-01.3).
    2. **Formality:** the target comes from the occasion (business 4, formal 5, casual 2, sport 1), plus formality_bias.
    3. **Weather:** use it only if observed_at is no more than 60 minutes old.
       - Feels-like below 5 °C → warmth 3, outerwear required. 5–15 °C → warmth ≥ 2, outerwear optional. Above 25 °C → warmth 1, no outerwear.
       - Chance of rain ≥ 50% → prefer water-resistant outerwear and shoes.
       - If the weather is missing or older than 60 min, skip these rules. The reason then says "Weather unavailable" and does not call the outfit weather-appropriate (C-01.2).
    4. **Shortlist:** for each category, keep garments that are available, not worn within repeat_window_days, and not tight. Score them: −|formality − target|, −|warmth − warmth target|, +1 for a favourite colour, −2 for an avoided colour. Keep the top 10.
    5. **Combine:** top × bottom (or a dress) × shoes, plus outerwear when required or optional. Combination score = sum of item scores + colour harmony (at most 2 non-neutral colours) + freshness (days since worn, capped at 14). Pick the highest score; on a tie, take the outfit worn least recently.
    6. **Reason (≤ 80 characters):** for example, "Class presentation, 8 °C and rain: wool coat, waterproof boots".
  - **Cost:** at most 10 × 10 × 10 × 10 = 10^4 combinations, under 50 ms. The weather call is cached for 15 min.
- **A5 Buy-or-skip verdict.**
  - **In:** a shop garment and its listing, FitEvaluation (A1, or "pending"), closet index, budget left.
  - **Out:** verdict buy or skip, with up to 3 reasons. The rules are applied in order, and each one that fires adds a reason:

1. No size is "good" → skip.
2. Price is above the budget left → skip.
3. The closet already has a near-duplicate (same category and colour name, formality ±1) → skip.
4. The closet has fewer than 3 items in this category, or none at the event formality → buy ("fills a gap").
5. Otherwise → buy if the fit is good and the price is within budget.

- **Cost:** O(closet) ≈ 300 comparisons.

- **A7 Look from a photo of a public figure.**
  1. Get images with ctx.images (I14c), up to 6.
  2. Detect pieces with vision.analyze (I15).
  3. For each category, keep the most frequent colour across the images.
  4. For each piece, find a closet match (same category, colour difference ΔE < 20). If there is none, search shop items with ctx.products (I14b), limited by category, colour, and max_price = budget left.

  Out: a DecisionView of kind look. **Cost:** dominated by I15, about 0.6 s per image.
- **Request handling:** a UserAction (I17) chooses the algorithm. "measure" calls I24. The result is saved as a Decision and sent to L8 as a DecisionView. Fig D2-3 shows A5's full path, including its errors.

Interface	Input	Output	Errors
I17 ui.action → decision.view	UserAction as in D0; action adds measure, ask_verdict, and recommend with occasion: enum, optional and location_name: string, optional (default: StylePreference location) [US-01]	DecisionView as in D0; reasons limited to 3	UNKNOWN_ACTION · STALE_VIEW (old view_id: L6 resends the current view)
GET /api/v1/decisions (phone)	kind: enum, optional · limit: int 1–50, default 20 · cursor: string, optional	200 {items: [{decision_id, kind, verdict, reasons, garment_ids, created_at}], next_cursor}	400 BAD_QUERY · 401 UNAUTHORIZED
POST /api/v1/spend (phone)	amount: decimal string > 0, required · currency: ISO 4217, required · listing_id: UUID, optional · spent_on: date, optional, default today	201 {spend_id, budget_left: decimal string}	400 VALIDATION_FAILED · 401 · 404 LISTING_NOT_FOUND
PUT /api/v1/preferences (phone)	StylePreference fields, all optional, with the ranges in Fig D2-4	200 StylePreference	400 VALIDATION_FAILED · 401

L7 Connectors — owner: Jace Schubert

Data model: ProviderAccount, CategoryRule, and CacheEntry (Fig D2-4). OAuth tokens are encrypted with a key kept in the secrets file, never in the database.

Behaviour (the same pipeline for every outside call):

1. Build the cache key (sha256 of provider + endpoint + normalised parameters). Return a fresh hit right away.
2. **Privacy filter:** each endpoint has an allowlist of outgoing parameters. Body data, images, and user ids never leave the house.
3. Take a token from the provider's rate-limit bucket (rate from configuration).
4. Call over HTTPS with a 3 s timeout. On 429, 5xx, or a timeout, retry after 1, 2, then 4 s, honouring Retry-After. On 401, refresh the token once.
5. Normalise the response into our models, cache it with a TTL (calendar 15 min, weather 15 min, products 6 h, images 7 days), and reply. Products the user opens are also sent to L4 (I16).

A6 Calendar categorisation.

- **In:** event title (string).
- **Out:** category ∈ {business, casual, formal, sport}.
- **Steps:** lower-case the title. Try the enabled CategoryRule patterns in priority order, user rules before default rules. The first match wins; no match → casual.
- **Default rules:** interview|client|presentation|meeting → business; wedding|gala|ceremony → formal; gym|run|yoga|match → sport.
- **Cost:** O(R) with R ≈ 30 rules.

Interface	Input	Output	Errors
I14a ctx.calendar_day	user_id: UUID, required · date: YYYY-MM-DD, required, today ± 7 days	{events: [{start, end: RFC 3339, title, category}], stale: bool, fetched_at}	NOT_LINKED · NEEDS_RELINK · PROVIDER_UNAVAILABLE (no cache) · BAD_QUERY
I14d ctx.weather [US-01]	lat, lon: float, required, rounded to 0.1° (about 10 km) before leaving the house	{temp_c, feels_like_c: float, precip_prob: int 0–100 %, condition: enum{clear, cloudy, rain, snow, storm, fog}, wind_kph: float, observed_at: RFC 3339, stale: bool}. Provider: Open-Meteo (no API key; free for non-commercial use).	PROVIDER_UNAVAILABLE — provider down and no cached reading · BAD_QUERY — coordinates out of range. A cached reading older than 60 min is still returned with stale = true; A4 treats it as "Weather unavailable".
I14b ctx.products	query: string 1–100, required · category, color, size_label: optional · max_price: decimal, optional · currency: ISO 4217 · limit: int 1–20, default 10	{products: [{sku, shop, title, price, currency, url, sizes, image_ref}], stale, fetched_at}	PROVIDER_UNAVAILABLE · RATE_LIMITED (local bucket empty for > 3 s) · BAD_QUERY
I14c ctx.images	query: person name, 1–60 chars, required · limit: int 1–6, default 6	{image_refs: URI[] (downloaded into the data store), stale, fetched_at}	PROVIDER_UNAVAILABLE · RATE_LIMITED · BAD_QUERY
Provider linking (phone)	POST /api/v1/providers/{provider}/link · GET …/callback?code&state · DELETE …/{provider}	201 {auth_url} · 302 to the phone page after the callback · 204	400 UNKNOWN_PROVIDER · 400 BAD_STATE (CSRF state mismatch) · 502 PROVIDER_REFUSED

L8 Presentation — owner: all (lead *[team: name one lead]*)

Data model: ConsentEvent (Fig D2-4). It records each allow, decline, or revoke with a session id but no user link and no image [US-04]. Session state lives in memory and is rebuilt from L6 after a restart; screen text and layout come from configuration.

Behaviour: the state machine in Fig D2-1, plus these steps:

1. When the Mirror connects, check its device token (Auth), then send the full ui.state (I6).
2. Each state change sends a new full state with seq + 1.
3. Voice commands (I5) are matched against a fixed list: next, back, select, buy, skip, try, measure, recommend, camera off, sleep. A command is accepted when its edit distance is ≤ 1 and confidence ≥ 0.6, and it becomes a UserAction (I17).
4. Health events (I21) and measurement progress (I25) update the banner and the T-pose guide.
5. **Camera consent [US-04]** (Fig D2-5):
   - The glass always shows "Camera off" or "Camera on".
   - Before try-on, measuring, or gestures, if capture is off, L8 asks "Allow the camera for this session?".
   - **Allow:** L8 sends capture.enable (I27) and shows "Camera on" only after L1 confirms.
   - **Decline, or no answer in 15 s:** L8 logs a ConsentEvent and leaves the camera off.
   - **Revoke:** "Camera off" (touch or voice) sends capture.disable.
   - While awake, L8 checks capture.status every 2 s. No reply, or a state that contradicts what L8 expects, means "Camera status unknown", and image functions are blocked (C-02.2).
6. **Profile unlock [US-05]:** after the user picks a profile, the keypad (digits shuffled) sends auth.unlock (I28). The session is bound to that user_id, and only that user's wardrobe, cards, and toasts are shown. Asleep ends the session.

Interface	Input	Output	Errors
I5 (in, WebSocket)	touch {x, y: int px, ts} · wake_word {ts} · voice_command {text ≤ 40, confidence 0–1}. No camera data [US-04].	—	Close 4401 — bad device token · 4409 — a second Mirror connection replaced this one
I6 ui.state (out, WebSocket)	—	ui_state/2 {seq: int, screen: enum (Fig D2-1 states), cards: Card[] ≤ 4, banner: {kind, text} | null, toast: string | null}	Close 4500 — server error; the Mirror reconnects every 2 s
I27 capture.enable / disable (out, to L1)	—	See L1	TIMEOUT or NO_SERVICE → state Unknown, "Camera status unknown" (Fig D2-5)
I18 mirror.control (out, JS bridge)	—	{cmd: enum{wake, sleep, brightness}, value: int 0–100?, cmd_id: UUID}; the Mirror replies {cmd_id, ok: bool}	No reply in 1 s: retry once, then log (glass state unknown)

Planes: data store and message bus — owner: Rehman Khalid

Data store — data model: the union of D1 Fig 1 and Fig D2-4, in one SQLite file (brain.db), with images under files/<garment_id>/.

Data store — behaviour:

- Settings: WAL mode, foreign_keys = ON, busy_timeout 2,000 ms.
- **Each table has exactly one writing component** (the owners in Fig D2-4 and D1); others read only.
- Schema migrations (Alembic) run at start-up.
- Nightly at 03:00: a .backup of the database plus a copy of the files to an external drive, keeping 7 copies.

Data store — interface: one repository module per owning component (Python functions; no shared SQL strings).

- **StorageBusy** (still locked after 2 s): callers map it to HTTP 503 or the bus error STORAGE_UNAVAILABLE.
- **StorageFull** (under 1 GB free): the store goes read-only and health reports DEGRADED.

Message bus — data model: no persistent data.

Message bus — behaviour: one broker process, busd. A ZeroMQ XSUB/XPUB proxy handles events, and a ROUTER socket handles request/reply by service name. Heartbeats every 1 s.

Message bus — interface:

- Every message uses one envelope: {schema: "name/version", msg_id: UUID, corr_id: UUID | null, sender: string, ts: int64 ns, body: object}.
- publish(topic, body) and request(service, body, timeout_ms) → body.
- Errors: TIMEOUT · NO_SERVICE (no worker registered) · TOO_LARGE (> 1 MB; send a reference instead) · BAD_SCHEMA (unknown schema name or version).

Glue: Authentication — owner: Jace Schubert *(proposed)*

Data model: Device and ProfileLock (Fig D2-4).

Behaviour (pairing steps):

1. On the glass, the user picks "pair phone". L8 asks Auth for a 6-digit code, valid for 120 s with at most 5 attempts.
2. The phone opens /pair and enters the code.
3. Auth creates a Device and returns a token dv_<device_id>.<32 random bytes, base64url>. The token is shown only once; only its argon2id hash is stored.
4. On each request, the device_id prefix finds the row and the hash is verified. Revoked devices fail.

The Mirror's own token is provisioned once at install.

Profile unlock and wardrobe separation [US-05] (steps):

1. Each user sets a 4–6 digit PIN from their own paired phone. Only its argon2id hash is stored, in ProfileLock.
2. On the glass, auth.unlock (I28) checks the PIN. It returns a session bound to that user_id that lasts until Asleep. After 5 wrong PINs the profile is locked for 5 min.
3. **Isolation rule:** every repository query takes user_id from the authenticated session or device token, never from the request body. Asking for another user's data returns 403 FORBIDDEN (HTTP) or the bus error FORBIDDEN.

Interface	Input	Output	Errors
auth.pair_code (bus, from L8)	{user_id: UUID, required}	{code: 6 digits, expires_at}	UNKNOWN_USER
I28 auth.unlock (bus, from L8)	{user_id: UUID, required; pin: 4–6 digits, required}	{session_id: UUID, user_id, expires_at}	WRONG_PIN (remaining attempts in fields) · LOCKED (until locked_until) · NO_PIN_SET — the user must set one from their phone
PUT /api/v1/profile/pin (phone)	pin: 4–6 digits, required; current_pin: required if a PIN is already set	204	400 VALIDATION_FAILED · 401 · 403 WRONG_CURRENT_PIN
I22 POST /api/v1/pair	code: 6 digits, required · device_name: 1–40 chars, required	201 {device_id: UUID, token: string (shown once), user_id}	404 CODE_UNKNOWN · 410 CODE_EXPIRED · 429 TOO_MANY_ATTEMPTS
GET / DELETE /api/v1/devices[/{id}]	Bearer token; id: UUID for delete	200 {items: [{device_id, kind, name, last_seen_at}]} · 204	401 UNAUTHORIZED · 404 NOT_FOUND · 409 CANNOT_REVOKE_SELF
verify(token) (in-process)	token: string	Device	Unauthorized (→ HTTP 401 / WebSocket close 4401)

Glue: Notifications — owner: Clare Hooper *(proposed)*

Data model: Notification (Fig D2-4).

Behaviour:

1. On notify.create, if an unread notification with the same (user_id, dedupe_key) exists, update it instead of adding a new one.
2. Publish notify.new; L8 shows a toast if that user is at the glass.
3. The phone page polls I26 every 60 s while it is open.
4. Expired rows are deleted daily.

The kinds are outfit_ready, relink_needed, measure_due (30 days after the last measurement), and system.

Interface	Input	Output	Errors
I20 notify.create (bus)	user_id: UUID, required · kind: enum, required · title: 1–60 chars · body: ≤ 200 chars · dedupe_key: string, optional · ttl_h: int 1–720, default 72	{notification_id: UUID, deduped: bool}	BAD_INPUT · UNKNOWN_USER
I26 GET /api/v1/notifications	unread: bool, optional · limit: 1–50, default 20 · cursor, optional	200 {items: [{notification_id, kind, title, body, created_at, read_at}], next_cursor}	400 BAD_QUERY · 401
POST /api/v1/notifications/{id}/read	id: UUID	204	401 · 404 NOT_FOUND

Glue: Configuration — owner: Rehman Khalid *(proposed)*

Data model: no persistent data in the database; configuration must still work when the database is the broken part. Three things on disk:

- /etc/scm/config.yaml: in git as config.example.yaml.
- /etc/scm/secrets.env: mode 600 and never committed. It holds the OAuth client secrets and the token encryption key.
- /var/lib/scm/config_history/ [US-03]: a numbered copy of every configuration that loaded successfully, plus index.json with {version, sha256, applied_at, known_good: bool}.

Key groups: capture, perception thresholds, A1 ease bands, A3 calibration, A4 weights, L7 rates and TTLs, timeouts, UI text.

Behaviour:

- **Start-up:** each process loads both files and validates them against a typed schema (Pydantic). If anything is invalid, it exits with a message naming the bad key, so systemd shows the cause.
- **Reload:** scmctl reload sends SIGHUP. The process validates again and applies only keys marked reloadable (thresholds, ease bands, rules). It then publishes I23. An invalid reload keeps the old configuration and logs an error.
- **Find and restore [US-03]:**
  - Every successful load saves a new version. A version becomes *known good* once every component has reported ok (I21) for 10 minutes.
  - scmctl status lists each component's health, so a maintainer can see which service failed.
  - scmctl config rollback restores the latest known-good version (or --to N), checks it, and restarts the affected services. The rollback is logged.

Interface	Input	Output	Errors
config.get (in-process)	key: dotted string	typed value	KeyError at start-up (never at run time, because the schema has defaults)
I23 config.changed (bus)	—	{version: int, changed_keys: string[]}	—
scmctl config check (CLI)	path	exit code 0, or 1 with a list of errors	—
scmctl status (CLI) [US-03]	—	one line per component: name, state (ok / degraded / down), since, detail	exit 2 if busd is unreachable ("health unknown")
scmctl config rollback (CLI) [US-03]	--to N: int, optional (default: latest known good)	exit 0, plus the restored version and the services restarted	exit 1 NO_KNOWN_GOOD · exit 1 INVALID_VERSION (fails the check; nothing changed) · exit 3 RESTART_FAILED (version restored, a service still down)

Glue: Logging and health — owner: Rehman Khalid *(proposed)*

Data model: no persistent data in the database. Logs go to journald, capped at 500 MB or 14 days.

Behaviour:

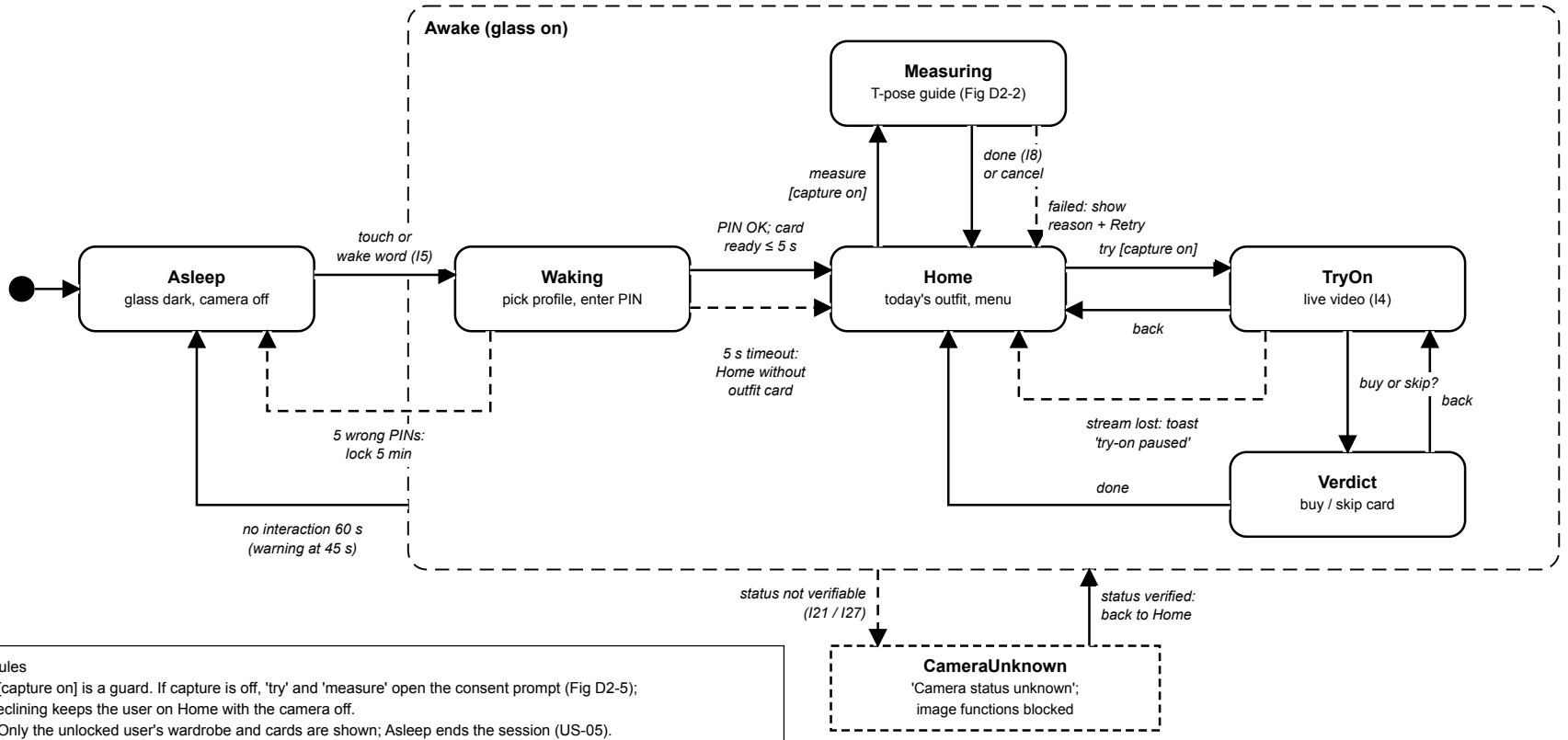
- **Log format:** every component logs JSON lines {ts, level, component, event, msg_id, corr_id, fields}. A corr_id follows one user request across layers.
- **Privacy rule:** never log images, measurements, tokens, or event titles.
- **Health:** every component publishes ops.health every 5 s. busd marks a component DOWN after 15 s of silence.
- **Restarts:** systemd restarts a crashed process, at most 5 times in 10 min.

Interface	Input	Output	Errors
log(event, **fields) (in-process)	event: snake_case string; fields: JSON-safe	—	Fields with blocked names are dropped and counted.
I21 ops.health (bus)	—	{component, state: enum{ok, degraded, down}, detail: ≤ 120 chars, metrics: {fps?, p95_ms?, queue?}}	—
GET /api/v1/health (phone)	—	200 {overall: enum, components: [{name, state, since}]}	401

Smart Closet Mirror — Fig D2-1: Mirror session state machine (L8 Presentation)

Goal: Turn a Gen 1 Mirror into a closet mirror that measures the user with a Kinect, shows their own clothes and shop items on their body, and tells them what to wear today and whether a new piece is worth buying.

Every transition is caused by a Mirror event (I5: touch or wake word), a user action (I17), or a capture-state event (I21, I27). L8 is the only component that changes these states.



Rules
- [capture on] is a guard. If capture is off, 'try' and 'measure' open the consent prompt (Fig D2-5); declining keeps the user on Home with the camera off.
- Only the unlocked user's wardrobe and cards are shown; Asleep ends the session (US-05).
- Socket loss to the WebView is handled inside the Mirror: it reconnects every 2 s (I6).

Legend

State
Error state
Start (first state on power-up)
Transition: trigger / action
Error or timeout transition
Group: an arrow leaving it applies to every state inside

Fig D2-1. Source: Design_Diagrams/D2_fig1_mirror_session.drawio.

Smart Closet Mirror — Fig D2-2: Body measurement state machine (L2 Perception, algorithm A3)

Goal: Turn a Gen 1 Mirror into a closet mirror that measures the user with a Kinect, shows their own clothes and shop items on their body, and tells them what to wear today and whether a new piece is worth buying.

Each state change publishes measure.progress (I25) {state, hint, progress 0–1} so L8 can draw the guide. 'Good frame' = all 20 joints with conf ≥ 0.6.

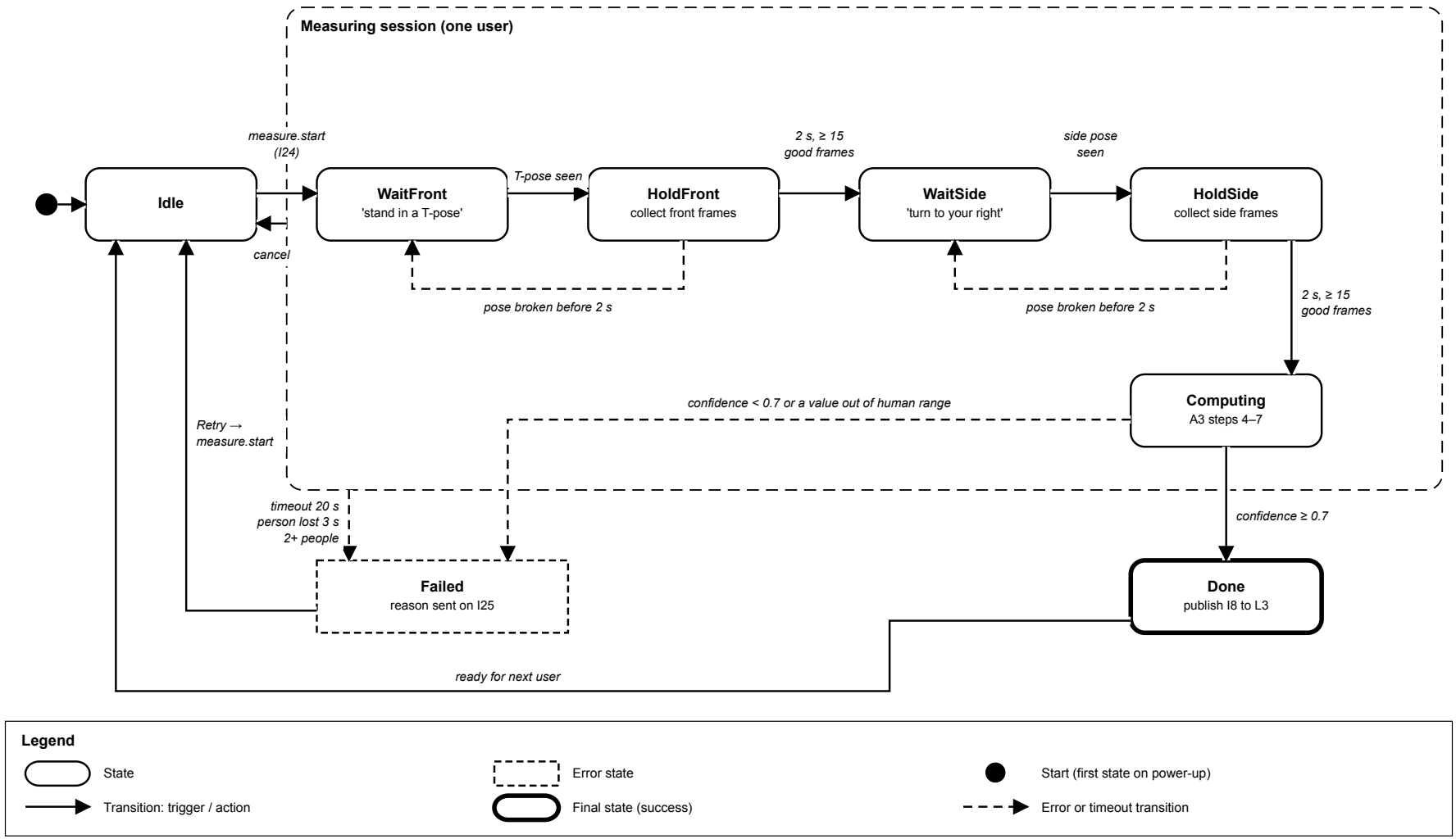


Fig D2-2. Source: Design_Diagrams/D2_fig2_measurement.drawio.

Smart Closet Mirror — Fig D2-3: 'Buy or skip?' decision flow with error paths

Goal: Turn a Gen 1 Mirror into a closet mirror that measures the user with a Kinect, shows their own clothes and shop items on their body, and tells them what to wear today and whether a new piece is worth buying.

Riskiest flow: it crosses L8, L6, L4, L5, L7 and the internet, under a 2 s budget (D0 Flow B). Row 1 runs left to right, row 2 right to left.

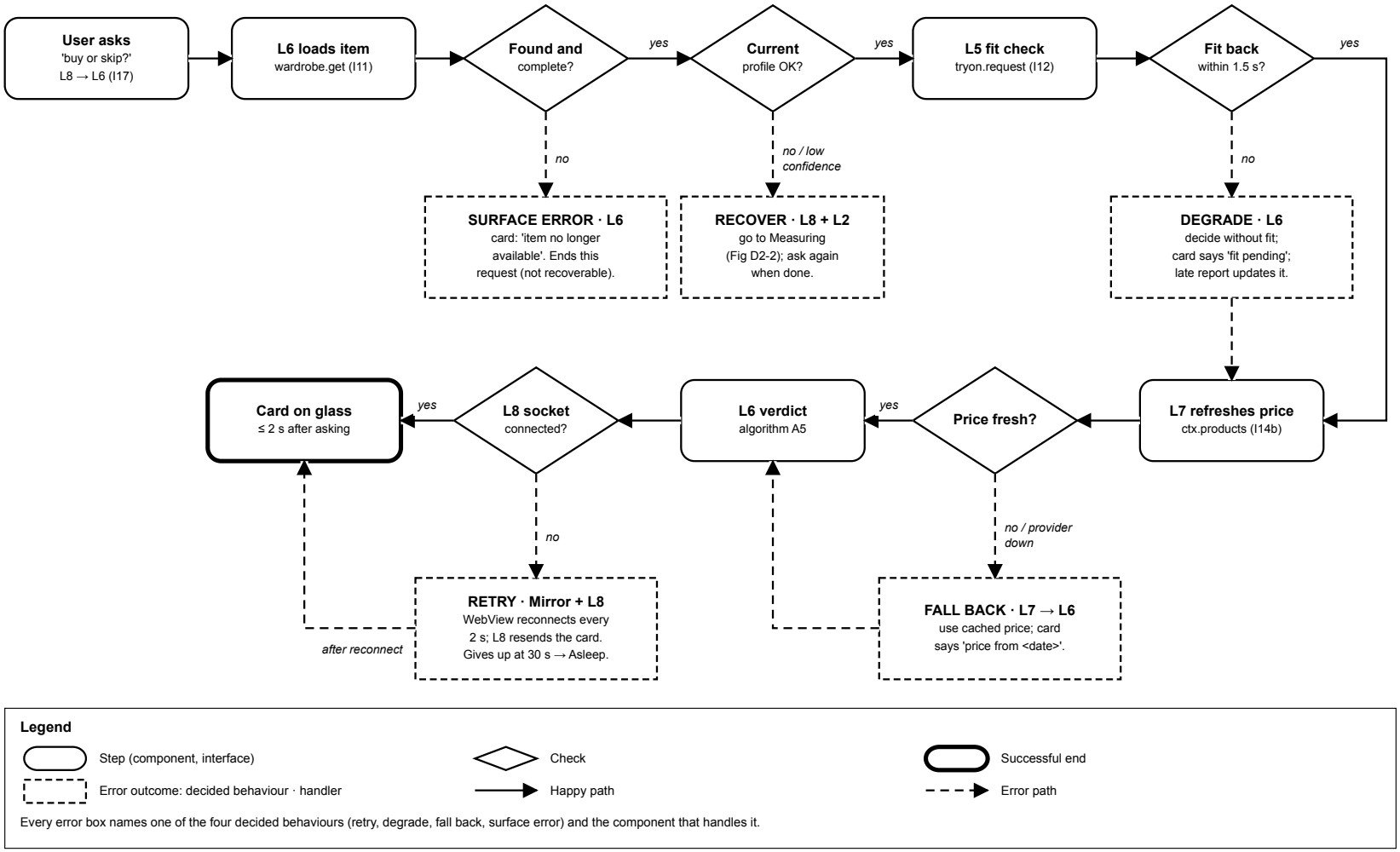


Fig D2-3, the figure required to show error paths. Source: Design_Diagrams/D2_fig3_buy_or_skip_errors.drawio.

Smart Closet Mirror — Fig D2-5: Camera capture consent state machine (L8 + L1, US-04)

Goal: Turn a Gen 1 Mirror into a closet mirror that measures the user with a Kinect, shows their own clothes and shop items on their body, and tells them what to wear today and whether a new piece is worth buying.

L8 owns the prompt and the on-glass indicator; L1 is the only component that can open the Kinect and does so only after capture.enable (I27).

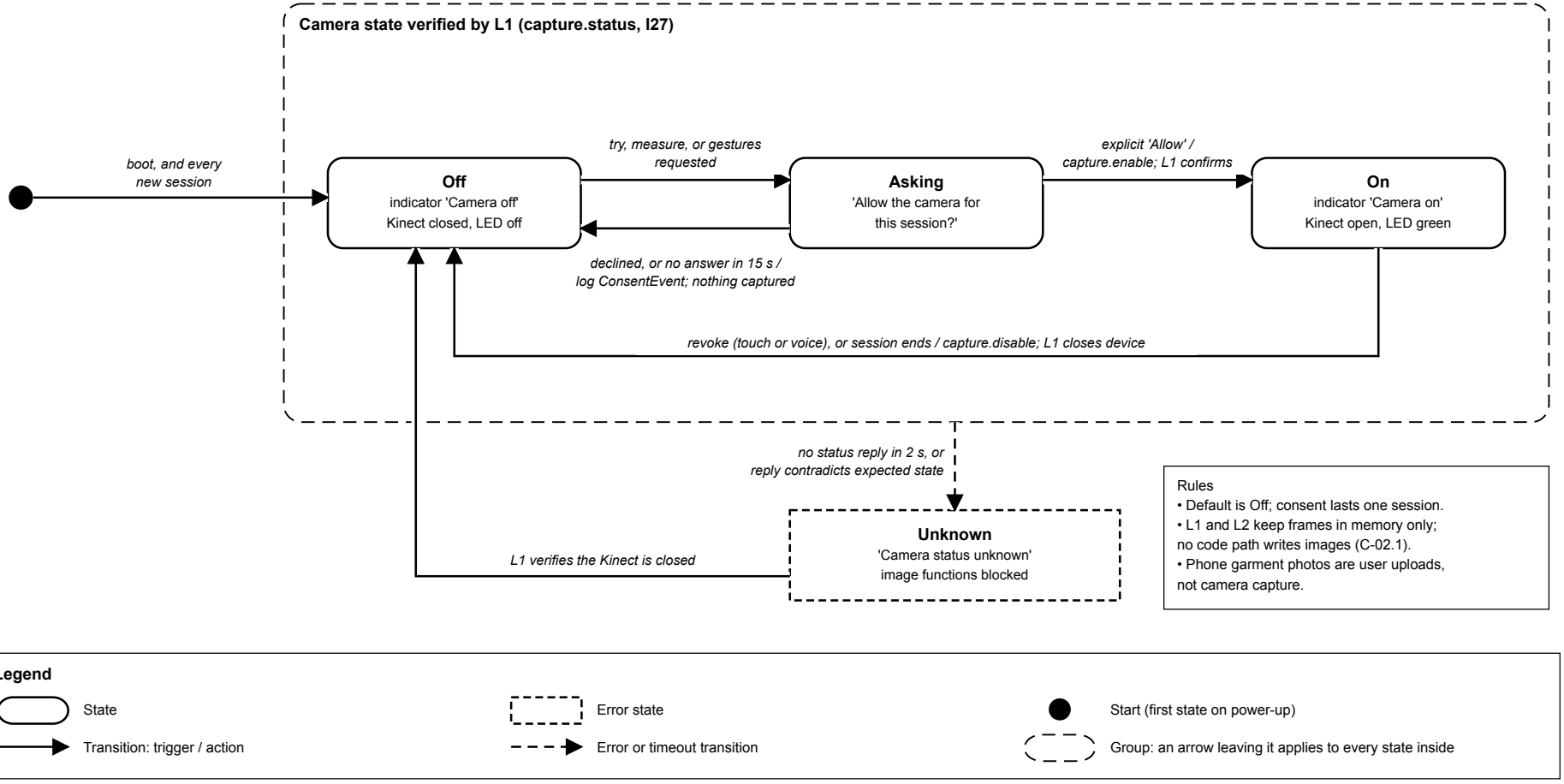


Fig D2-5 [US-04, UC-02]. Source: Design_Diagrams/D2_fig5_camera_consent.drawio.

Smart Closet Mirror — Fig D2-6: 'Recommend an outfit' flow (UC-01, algorithm A4) with error paths

Goal: Turn a Gen 1 Mirror into a closet mirror that measures the user with a Kinect, shows their own clothes and shop items on their body, and tells them what to wear today and whether a new piece is worth buying.

Covers acceptance criteria C-01.1 (main), C-01.2 (weather unavailable), and C-01.3 (incomplete wardrobe). Budget: card ≤ 5 s after the request.

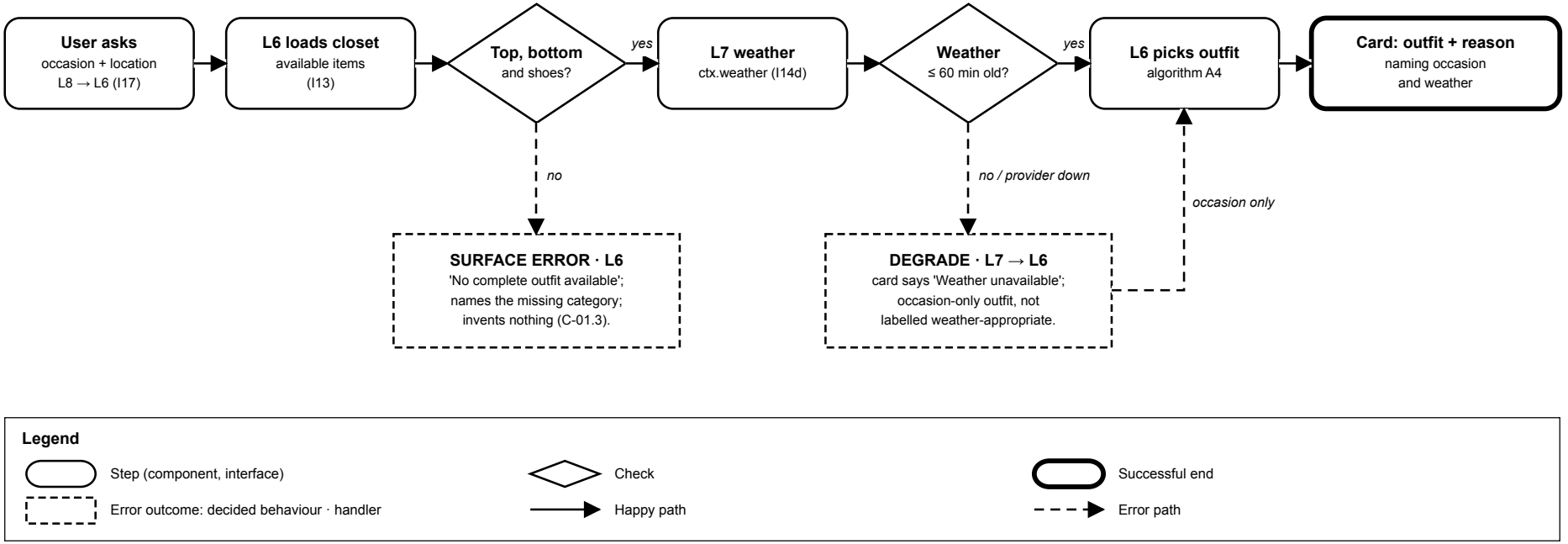


Fig D2-6 [US-01, UC-01]. Source: Design_Diagrams/D2_fig6_outfit_errors.drawio.

Smart Closet Mirror — Fig D2-4: Data model for the components finished in D2

Goal: Turn a Gen 1 Mirror into a closet mirror that measures the user with a Kinect, shows their own clothes and shop items on their body, and tells them what to wear today and whether a new piece is worth buying.

Owners: Device, ProfileLock = Auth · ConsentEvent = L8 · Notification = Notifications · BodyModel = L3 · Decision, StylePreference, SpendEntry = L6 · ProviderAccount, CategoryRule, CacheEntry = L7.

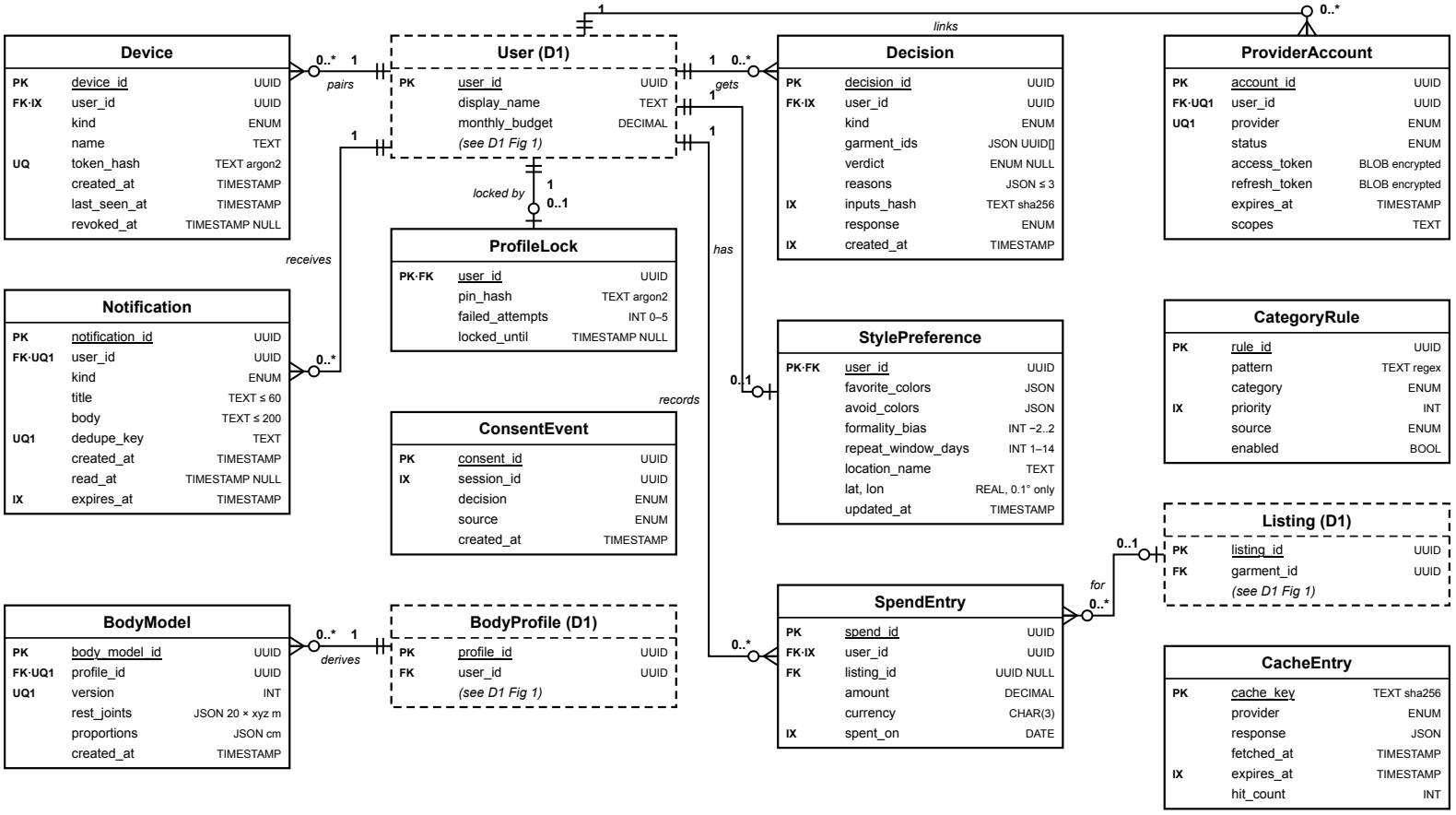


Fig D2-4. Source: Design_Diagrams/D2_fig4_data_model.drawio.

3. Edge cases and error handling

Team-written section. The course policy requires the edge-case analysis to be the team's own work, so the rows below are a grid to fill in, not answers. The grid gives every component one row for each of the four categories: **B** boundary values, **T** timing and concurrency, **I** bad or missing input, **X** external dependencies. Add more rows as needed (EC-L6-I2, and so on). Every row needs a decided behaviour (retry, degrade, fall back, or surface error), a named handler, and whether it is recoverable. Figs D2-3 and D2-6 are the error-path diagrams, and D2-5 shows the camera-consent failures; cite their boxes where they apply.

ID	Component	Category	Edge case	Design decision	Handler	Recoverable?
EC-L1-B	L1 Capture	Boundary values				
EC-L1-T	L1 Capture	Timing / concurrency				
EC-L1-I	L1 Capture	Bad or missing input				
EC-L1-X	L1 Capture	External dependency				
EC-L2-B	L2 Perception	Boundary values				
EC-L2-T	L2 Perception	Timing / concurrency				
EC-L2-I	L2 Perception	Bad or missing input				
EC-L2-X	L2 Perception	External dependency				
EC-L3-B	L3 Body	Boundary values				
EC-L3-T	L3 Body	Timing / concurrency				
EC-L3-I	L3 Body	Bad or missing input				
EC-L3-X	L3 Body	External dependency				
EC-L4-B	L4 Wardrobe	Boundary values				

EC-L4-T	L4 Wardrobe	Timing / concurrency				
EC-L4-I	L4 Wardrobe	Bad or missing input				
EC-L4-X	L4 Wardrobe	External dependency				
EC-L5-B	L5 Try-on	Boundary values				
EC-L5-T	L5 Try-on	Timing / concurrency				
EC-L5-I	L5 Try-on	Bad or missing input				
EC-L5-X	L5 Try-on	External dependency				
EC-L6-B	L6 Decision	Boundary values				
EC-L6-T	L6 Decision	Timing / concurrency				
EC-L6-I	L6 Decision	Bad or missing input				
EC-L6-X	L6 Decision	External dependency				
EC-L7-B	L7 Connectors	Boundary values				
EC-L7-T	L7 Connectors	Timing / concurrency				
EC-L7-I	L7 Connectors	Bad or missing input				
EC-L7-X	L7 Connectors	External dependency				
EC-L8-B	L8 Presentation	Boundary values				
EC-L8-T	L8 Presentation	Timing / concurrency				

EC-L8-I	L8 Presentation	Bad or missing input				
EC-L8-X	L8 Presentation	External dependency				
EC-DS-B	Data store	Boundary values				
EC-DS-T	Data store	Timing / concurrency				
EC-DS-I	Data store	Bad or missing input				
EC-DS-X	Data store	External dependency				
EC-BUS-B	Message bus	Boundary values				
EC-BUS-T	Message bus	Timing / concurrency				
EC-BUS-I	Message bus	Bad or missing input				
EC-BUS-X	Message bus	External dependency				
EC-AUTH-B	Authentication	Boundary values				
EC-AUTH-T	Authentication	Timing / concurrency				
EC-AUTH-I	Authentication	Bad or missing input				
EC-AUTH-X	Authentication	External dependency				
EC-NOTIF-B	Notifications	Boundary values				
EC-NOTIF-T	Notifications	Timing / concurrency				
EC-NOTIF-I	Notifications	Bad or missing input				

EC-NOTIF-X	Notifications	External dependency				
EC-CFG-B	Configuration	Boundary values				
EC-CFG-T	Configuration	Timing / concurrency				
EC-CFG-I	Configuration	Bad or missing input				
EC-CFG-X	Configuration	External dependency				
EC-LOG-B	Logging and health	Boundary values				
EC-LOG-T	Logging and health	Timing / concurrency				
EC-LOG-I	Logging and health	Bad or missing input				
EC-LOG-X	Logging and health	External dependency				

4. Requirements traceability table

Team-written section. One row per user story or requirement, functional (F) and non-functional (NF). Rows US-01 to US-05 are copied from User_Stories (Week 4); add NF rows for the measurable targets in its acceptance criteria (for example, weather data no older than 60 minutes, and zero image files written after consent is declined). Cite specific design elements from the catalogue below (an entity, endpoint, algorithm, interface, or figure, not just a component name), and cite edge-case IDs from part 3. Explain every gap; no blank cells.

Req ID	Requirement (short)	F / NF	Priority	Design elements	How they satisfy it	Edge cases · figures
US-01	Outfit suggestions from owned clothing, the occasion, and current weather (UC-01; C-01.1–C-01.3)					
US-02	Add, correct, and remove clothing items in my personal inventory					
US-03	Maintainer can identify a failed service and restore the last working configuration					
US-04	Know whether the camera is active and decline image capture (UC-02; C-02.1–C-02.3)					
US-05	Access only my own wardrobe and preferences on a shared mirror					
NF-01						
NF-02						
NF-03						

Reverse check: component → requirements

Component	Requirements it serves (IDs)	If none: keep (why) or cut
L1 Capture		
L2 Perception		
L3 Body		
L4 Wardrobe		
L5 Try-on		

L6 Decision		
L7 Connectors		
L8 Presentation		
Data store, message bus		
Authentication		
Notifications		
Configuration		
Logging and health		

Design element catalogue (for citing in parts 3 and 4)

Kind	Elements and where they are defined
Entities	**D1 Fig 1:** User, BodyProfile, Garment, GarmentSize, GarmentAsset, Listing, Outfit, OutfitItem, WearEvent, FitResult. **Fig D2-4:** Device, ProfileLock, ConsentEvent, Notification, BodyModel, Decision, StylePreference, SpendEntry, ProviderAccount, CategoryRule, CacheEntry. **Added attributes:** Garment.warmth, Garment.water_resistant.
Algorithms	**A1** fit evaluation (D1 §3) · **A2** live garment overlay (D1 §3) · **A3** body measurement (L2, Fig D2-2) · **A4** outfit recommendation (L6, Fig D2-6) · **A5** buy-or-skip verdict (L6, Fig D2-3) · **A6** calendar categorisation (L7) · **A7** look from a photo (L6)
HTTP endpoints	**D1 §5:** POST / GET / PATCH / DELETE /api/v1/garments. **D2:** GET /api/v1/decisions · POST /api/v1/spend · PUT /api/v1/preferences · /api/v1/providers/* · POST /api/v1/pair · PUT /api/v1/profile/pin · /api/v1/devices · /api/v1/notifications · GET /api/v1/health
Bus methods and events	wardrobe.get (I11) · tryon.request (I12) · vision.analyze (I15) · ctx.calendar_day / products / images / weather (I14a–d) · capture.enable / disable (I27) · auth.unlock (I28) · measure.start / cancel (I24) · measure.progress (I25) · body.current · capture.status · auth.pair_code · notify.create (I20) · ops.health (I21) · config.changed (I23) · intent.gesture (I19)
Interfaces	I1–I17 (D0 §4, revised above) · I18–I28 (D2)
Figures	D0: block diagram, data flow (Flows A, B). D1: Fig 1 data model. D2: Fig D2-1 session, D2-2 measurement, D2-3 buy-or-skip error paths, D2-4 data model, D2-5 camera consent, D2-6 outfit error paths.
Timing budgets	Motion → glass ≤ 250 ms (D0 Flow A; A2 30 ms, encode 25 ms) · verdict ≤ 2 s (D0 Flow B, Fig D2-3) · outfit card ≤ 5 s after the request (Figs D2-1, D2-6) · weather no older than 60 min (A4, C-01.2) · fit reply ≤ 1.5 s (I12)

5. Design rationale

Team-written section. Write 6–10 entries in the required form. Mark any decision the peer review changed, giving its action item ID from part 6. The D0 decision log (D1–D7: brain PC, Kinect 360, H.264 video, single L7, message bus, voice in L8, clip-over bracket) is referenced by number, not repeated.

#	We chose ___ over ___ because ___.	We accepted the trade-off of ___ because ___.	Changed by review? (item ID)
R1			
R2			
R3			
R4			
R5			
R6			
R7			
R8			

Decisions in D1 and D2 that may be worth an entry (pick the significant ones; the reasoning is yours):

- SQLite vs PostgreSQL (D1 §6)
- 2-D skinned warp vs 3-D cloth or neural try-on (A2)
- Ease bands vs ML fit prediction (A1)
- Front + side depth ellipse vs a single front view (A3)
- MediaPipe Pose on Kinect RGB vs OpenNI/NiTE skeleton tracking
- Splitting I14, I15, and I6 (interface revisions)
- Dropping the 3-D avatar mesh from I9
- Device pairing tokens vs user accounts with passwords
- In-app notifications vs web push
- One writer per table
- JSON snapshot of garment_ids in Decision
- Wake by touch or voice vs presence detection from a camera (US-04)
- Consent per session vs remembered consent (US-04)
- Profile PIN vs phone unlock vs face recognition (US-05)
- Open-Meteo vs a paid weather API (US-01)
- File-based configuration history vs storing it in the database (US-03)

6. Peer design review notes

Team-written section, from the in-class review. The same table is in Design_Diagrams/Peer_Review_Notes.md; keep the two in step. Add one block of rows per reviewing team. Priorities: must-fix, worth considering, out of scope for now. Apply the must-fix items to D2 before submitting and mark them in part 5.

ID	Criterion	Issue	Raised by (team)	Action	Owner	Priority	Due
PR-01							
PR-02							
PR-03							
PR-04							
PR-05							
PR-06							

Summary

Criterion that drew the most feedback: *[completeness / feasibility / clarity / risk, and why]*

Top two issues we raised while reviewing other teams:

1. *[team reviewed, issue, criterion]*
2. *[team reviewed, issue, criterion]*